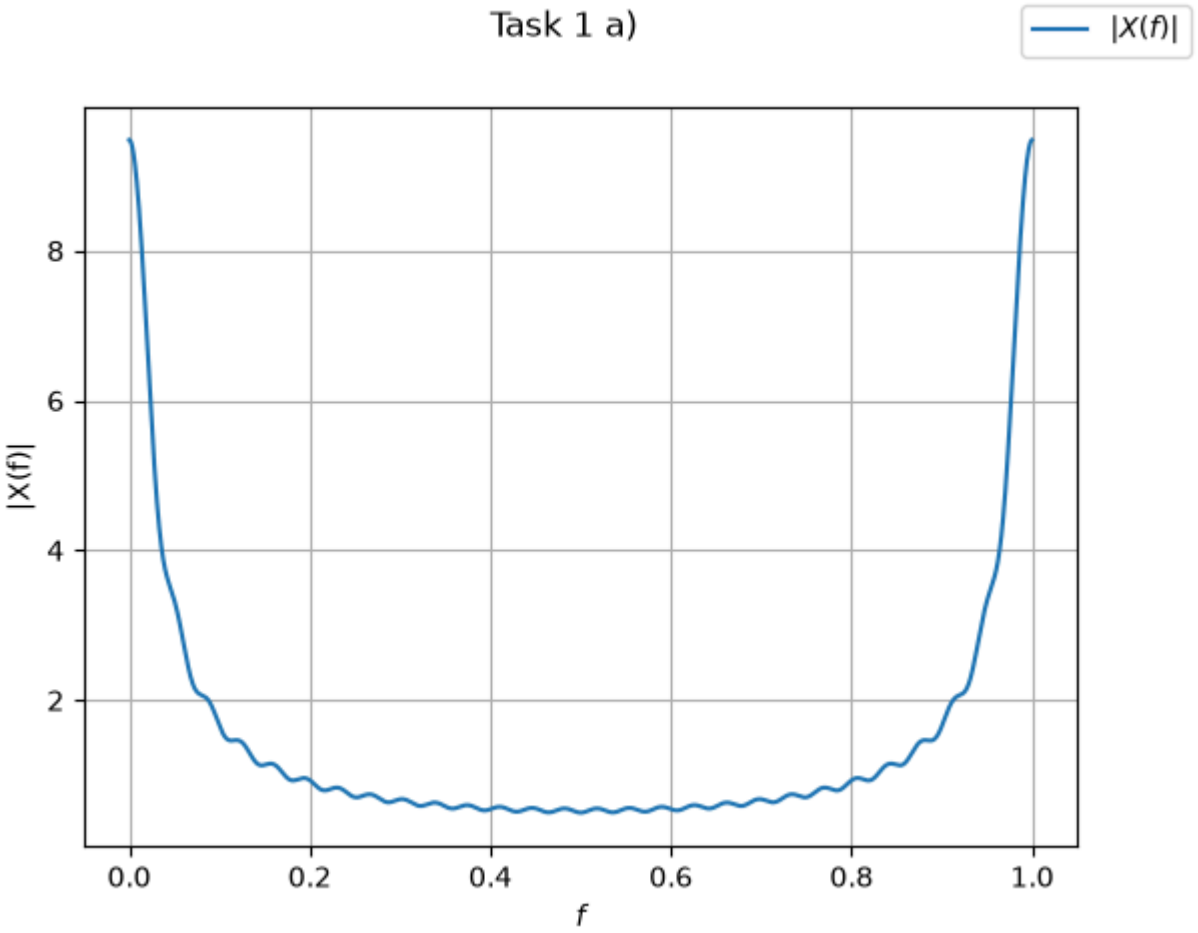
Problem 1)

a)



b)

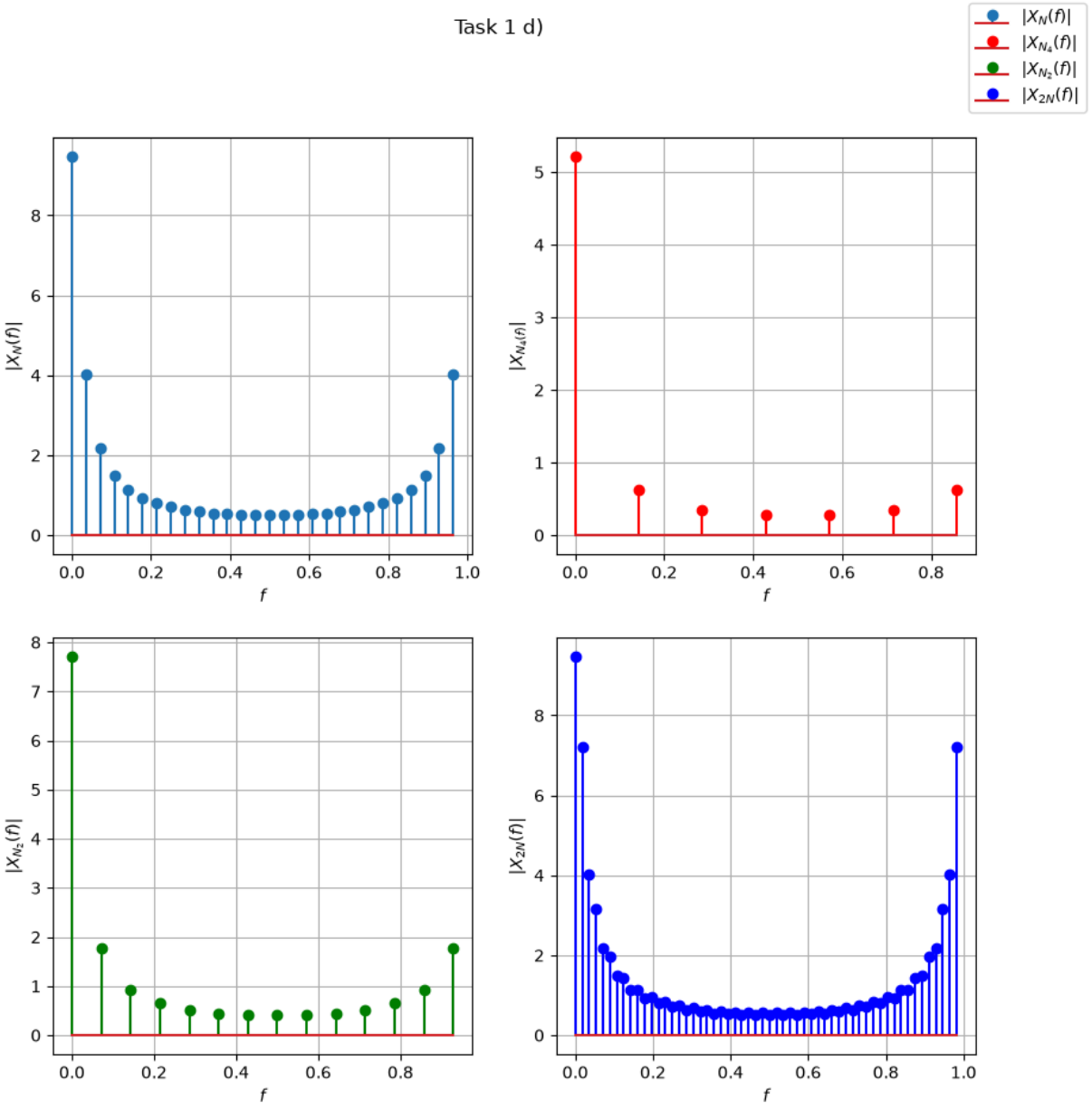
PYTHON

```
X_N = fft(x, N_x)
X_N_4 = fft(x, int(N_x/4))
X_N_2 = fft(x, int(N_x/2))
X_2N = fft(x, 2*N_x)
```

c)

f_N is 0.03571428571428571
f_{N_4} is 0.14285714285714285
f_{N_2} is 0.07142857142857142
f_{2N} is 0.017857142857142856

d)

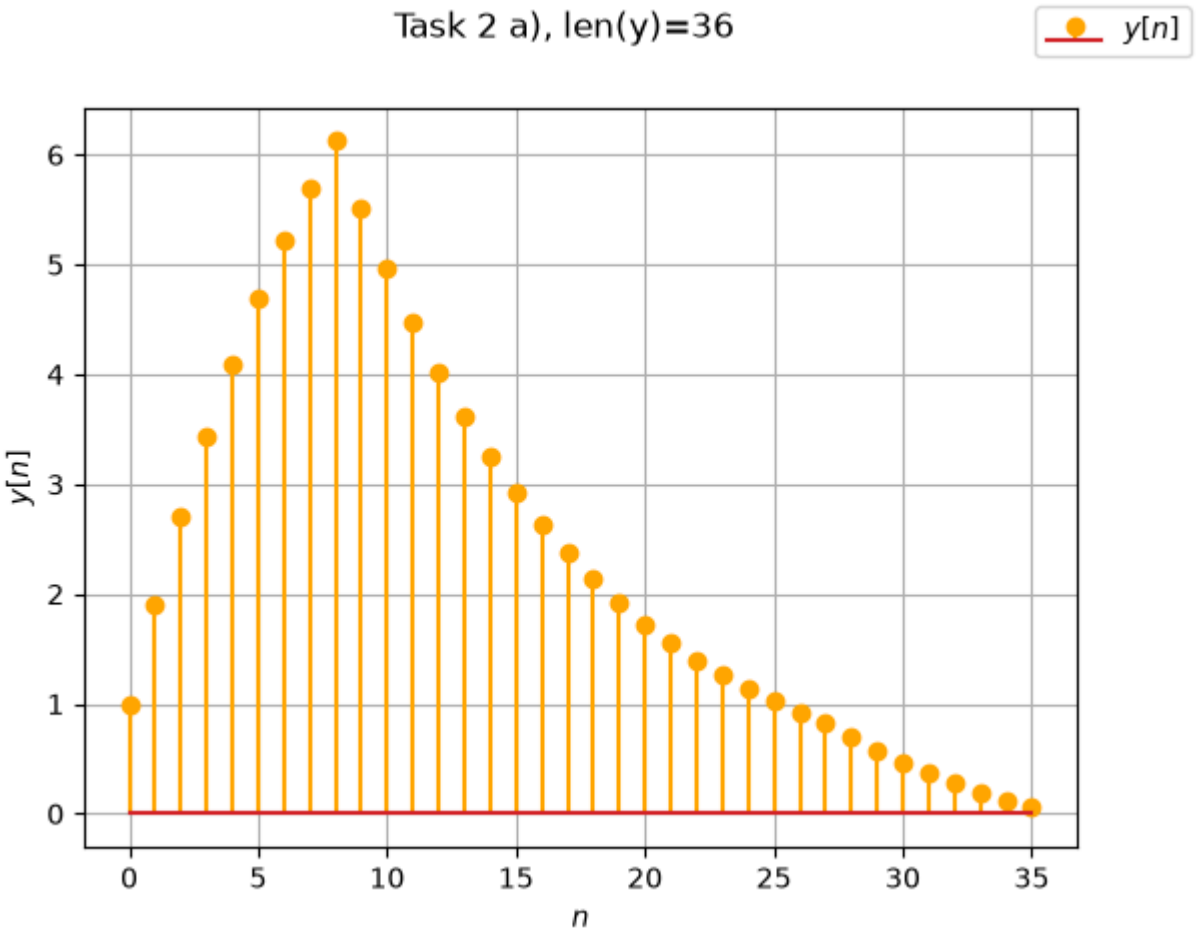


e)

Because of the sampling theorem, as long as we have enough samples, then any real signal will be causal and so it wont have any negative frequencies, or we wont care about them as all the information we need is in the frequency range $f \in [0, 0.5]$.

Problem 2)

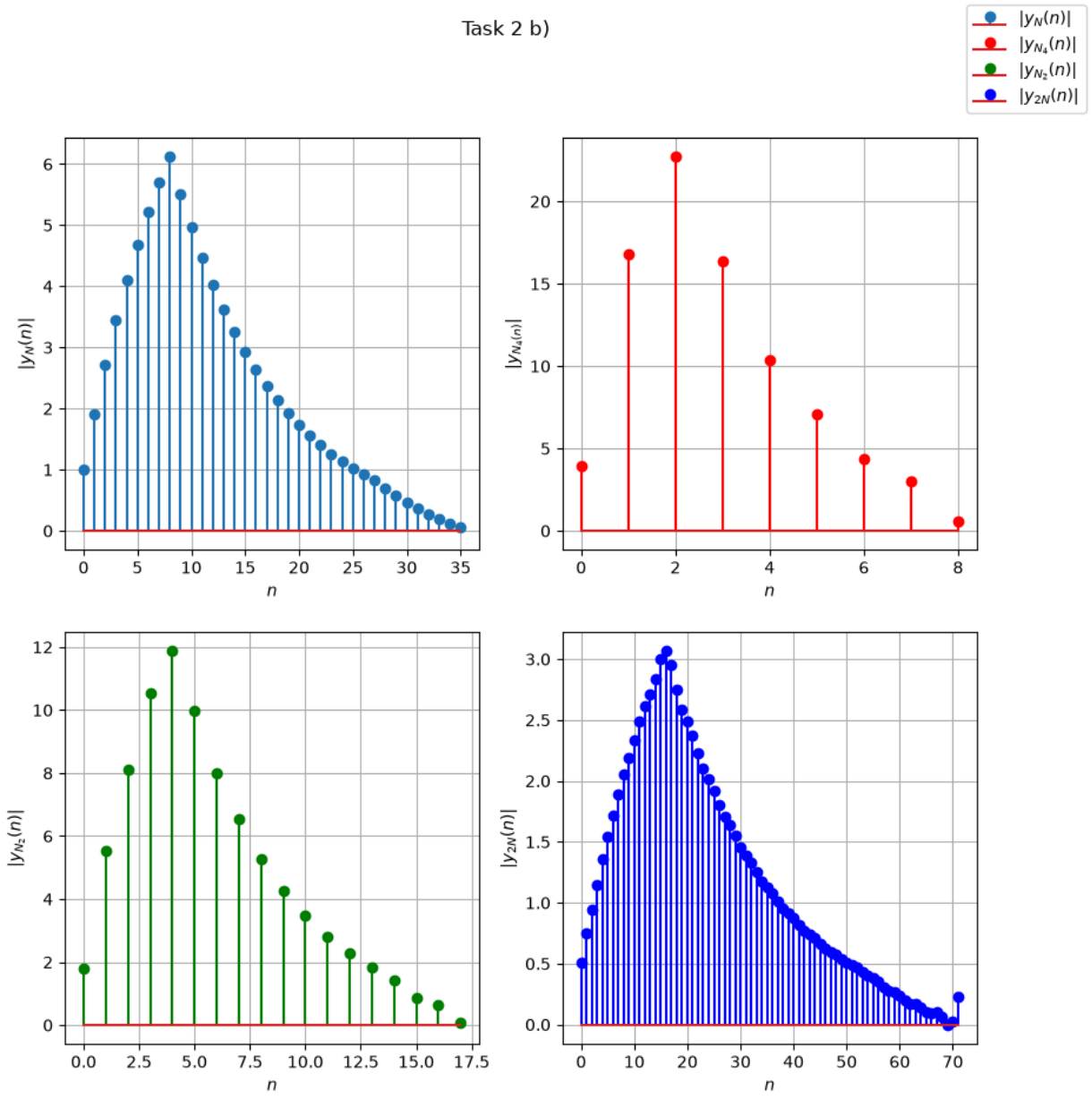
a)



It agrees with the theory that the length of signal $y[n]$ from a signal $x[n]$ of length $L = 28$ through a filter $h[n]$ of length $M = 9$ is:

$$N_y = L + M - 1 = 28 + 9 - 1 = 36$$

b)



When I used fft and ifft instead of rfft and irfft, then for the 2N length version I got a form of aliasing error where the padded zeros added from ifft made it so what was supposed to be the negative frequencies from the dft at the end came into the middle. This created a layer of frequencies over the entire graph. But switching it with the rfft and irfft, I get the correct output, where we can see that the lower sample amount, the less similar the output signal is to the one in a), while the cyan one with equal sample length to N_y is identical, and the dark blue one is twice the resolution (with one wrong sample at the end).

Code:

PYTHON

```python
import matplotlib
matplotlib.use("QtAgg")
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import freqz, fftconvolve
from scipy.fft import fft, rfft, irfft

# Problem 1
# a)

N_x = 28
n = np.arange(0, N_x)
x = 0.9**n

w, X = freqz(x, worN=2048, whole=True)

f = w / (2 * np.pi)

mag_X = abs(X) / 2048

fig1, ax1 = plt.subplots(1)
ax1.plot(f, mag_X, label=r"$|X(f)|$",)
ax1.grid()
ax1.set_xlabel(r"$f$")
ax1.set_ylabel("|X(f)|")
fig1.legend()
fig1.suptitle("Task 1 a)")




# b)
X_N = fft(x, N_x)
X_N_4 = fft(x, int(N_x/4))
X_N_2 = fft(x, int(N_x/2))
X_2N = fft(x, 2*N_x)

# c)

# Relationship between DFT index k and normalized frequency f is f = k / N
# f for k=1:

f_N = 1 / N_x
f_N_4 = 1 / (N_x/4)
f_N_2 = 1 / (N_x/2)
f_2N = 1 / (2*N_x)

print(r"$f_N$ is", f_N)
print(r"$f_{N_4}$ is", f_N_4)
print(r"$f_{N_2}$ is", f_N_2)
print(r"$f_2N$ is", f_2N)
```

```
# d)


mag_X_N = np.abs(X_N) * f_N
mag_X_N_4 = np.abs(X_N_4) * f_N_4
mag_X_N_2 = np.abs(X_N_2) * f_N_2
mag_X_2N = np.abs(X_2N) * f_2N

f_n = np.arange(0, 1, f_N)
f_n_4 = np.arange(0, 1, f_N_4)
f_n_2 = np.arange(0, 1, f_N_2)
f_2n = np.arange(0, 1, f_2N)

fig2, ((ax2, ax3), (ax4, ax5)) = plt.subplots(2,2, figsize=(10,10))
ax2.stem(f_n, mag_X_N, label=r"$|X_N(f)|$",)
ax2.grid()
ax2.set_xlabel(r"$f$")
ax2.set_ylabel(r"$|X_N(f)|$")


ax3.stem(f_n_4, mag_X_N_4, label=r"$|X_{N_4}(f)|$", linefmt="r")
ax3.grid()
ax3.set_xlabel(r"$f$")
ax3.set_ylabel(r"$|X_{N_4(f)}|$")


ax4.stem(f_n_2, mag_X_N_2, label=r"$|X_{N_2}(f)|$", linefmt="g")
ax4.grid()
ax4.set_xlabel(r"$f$")
ax4.set_ylabel(r"$|X_{N_2}(f)|$")


ax5.stem(f_2n, mag_X_2N, label=r"$|X_{2N}(f)|$",linefmt="b")
ax5.grid()
ax5.set_xlabel(r"$f$")
ax5.set_ylabel(r"$|X_{2N}(f)|$")

fig2.legend()
fig2.suptitle("Task 1 d)")


# Problem 2

# a)

N_h = 9

h_n = np.ones(N_h)

y_n = np.convolve(x, h_n, "full")
N_y = N_x + N_h - 1
```

```
n_y = np.arange(0, N_y)

fig3, ax6 = plt.subplots(1)
ax6.stem(n_y, y_n, label=r"$y[n]$", linefmt="orange")
ax6.grid()
ax6.set_xlabel(r"$n$")
ax6.set_ylabel(r"$y[n]$")
fig3.legend()
fig3.suptitle("Task 2 a), len(y)=36")

# b)

X_f = rfft(x, N_y)

H_f = rfft(h_n, N_y)

Y_f = X_f * H_f

#Y_f = fftconvolve(x, h_n, mode="full")


y_n = np.arange(0, N_y)
y_n_4 = np.arange(0, int(N_y/4))
y_n_2 = np.arange(0, int(N_y/2))
y_2n = np.arange(0, 2*N_y)



y_N = irfft(Y_f, n=N_y)
y_N_4 = irfft(Y_f, n=int(N_y/4))
y_N_2 = irfft(Y_f, n=int(N_y/2))
y_2N = irfft(Y_f, n=2*N_y)

fig4, ((axa, axb), (axc, axd)) = plt.subplots(2,2, figsize=(10,10))
axa.stem(y_n, y_N, label=r"$|y_N(n)|$",)
axa.grid()
axa.set_xlabel(r"$n$")
axa.set_ylabel(r"$|y_N(n)|$")


axb.stem(y_n_4, y_N_4, label=r"$|y_{N_4}(n)|$", linefmt="r")
axb.grid()
axb.set_xlabel(r"$n$")
axb.set_ylabel(r"$|y_{N_4(n)}|$")


axc.stem(y_n_2, y_N_2, label=r"$|y_{N_2}(n)|$", linefmt="g")
axc.grid()
axc.set_xlabel(r"$n$")
axc.set_ylabel(r"$|y_{N_2}(n)|$")


axd.stem(y_2n, y_2N, label=r"$|y_{2N}(n)|$",linefmt="b")
```

```
    axd.grid()
    axd.set_xlabel(r"$n$")
    axd.set_ylabel(r"$|y_{2N}(n)|$")

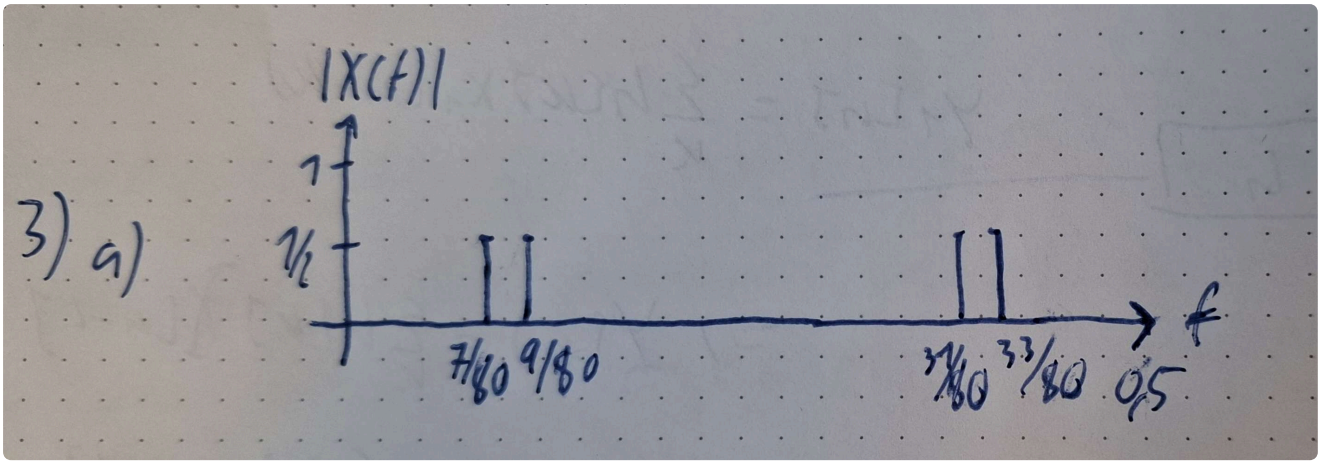
    fig4.legend()
    fig4.suptitle("Task 2 b)")



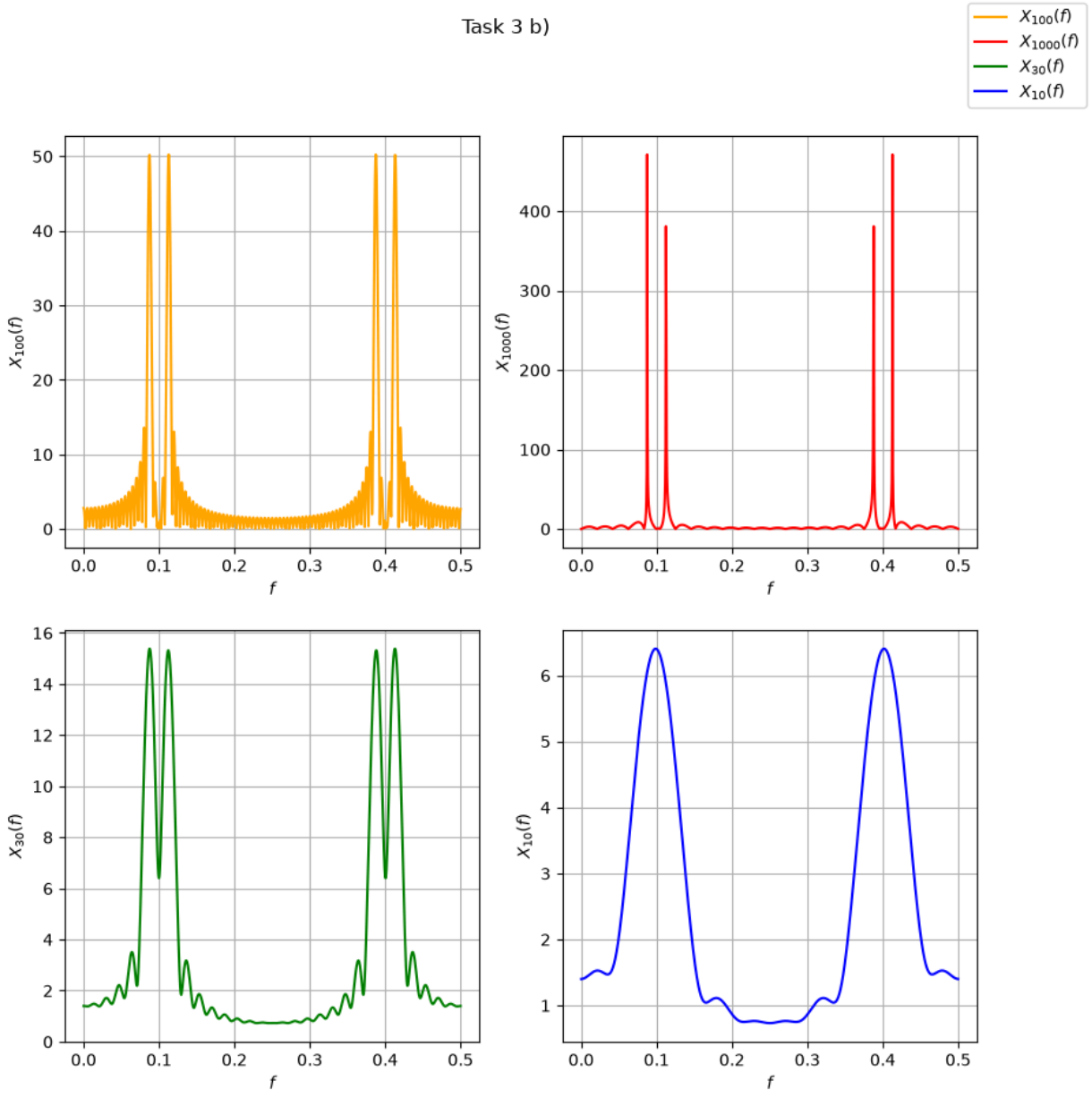
    plt.show()
```

Problem 3)

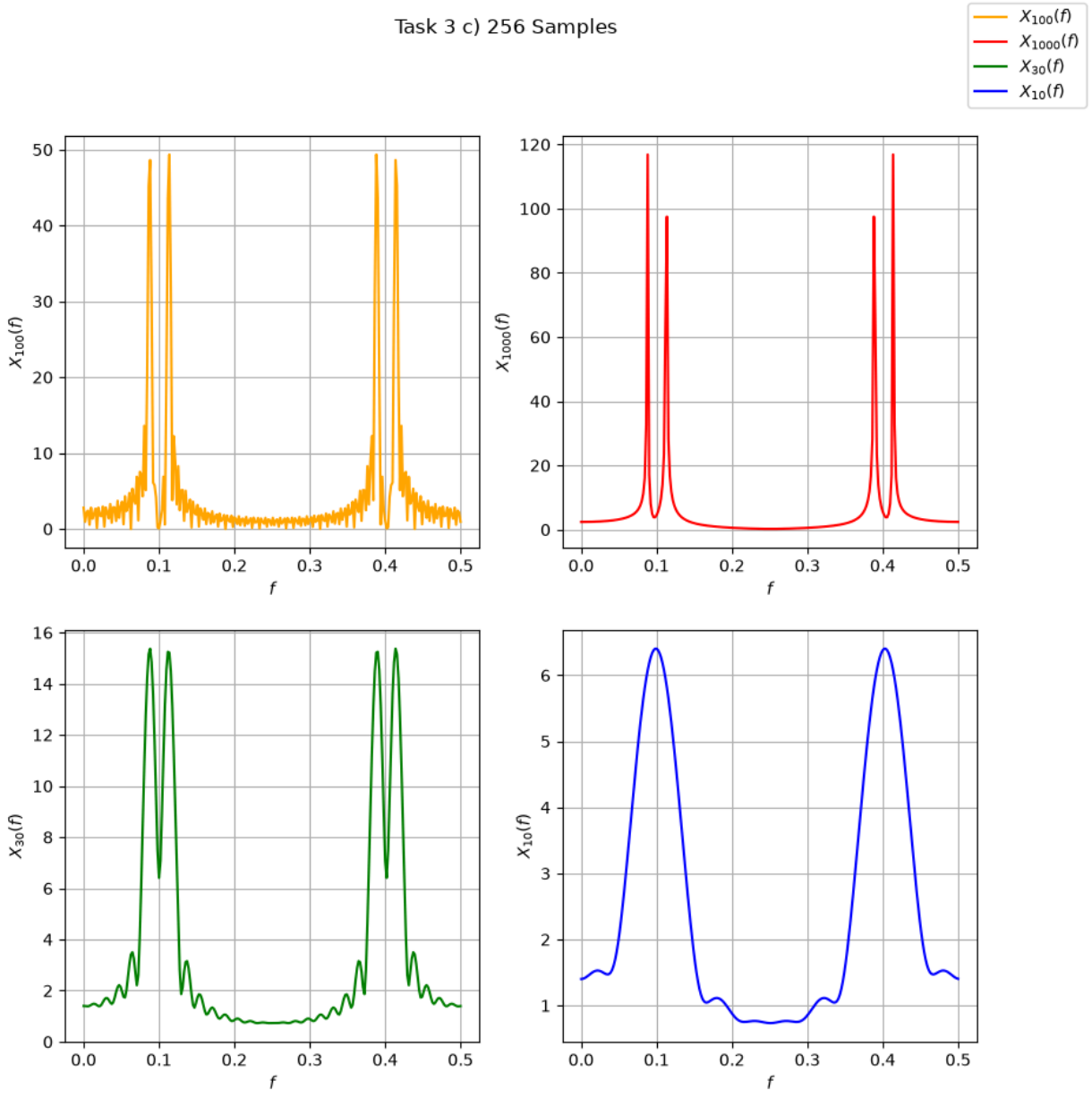
a)

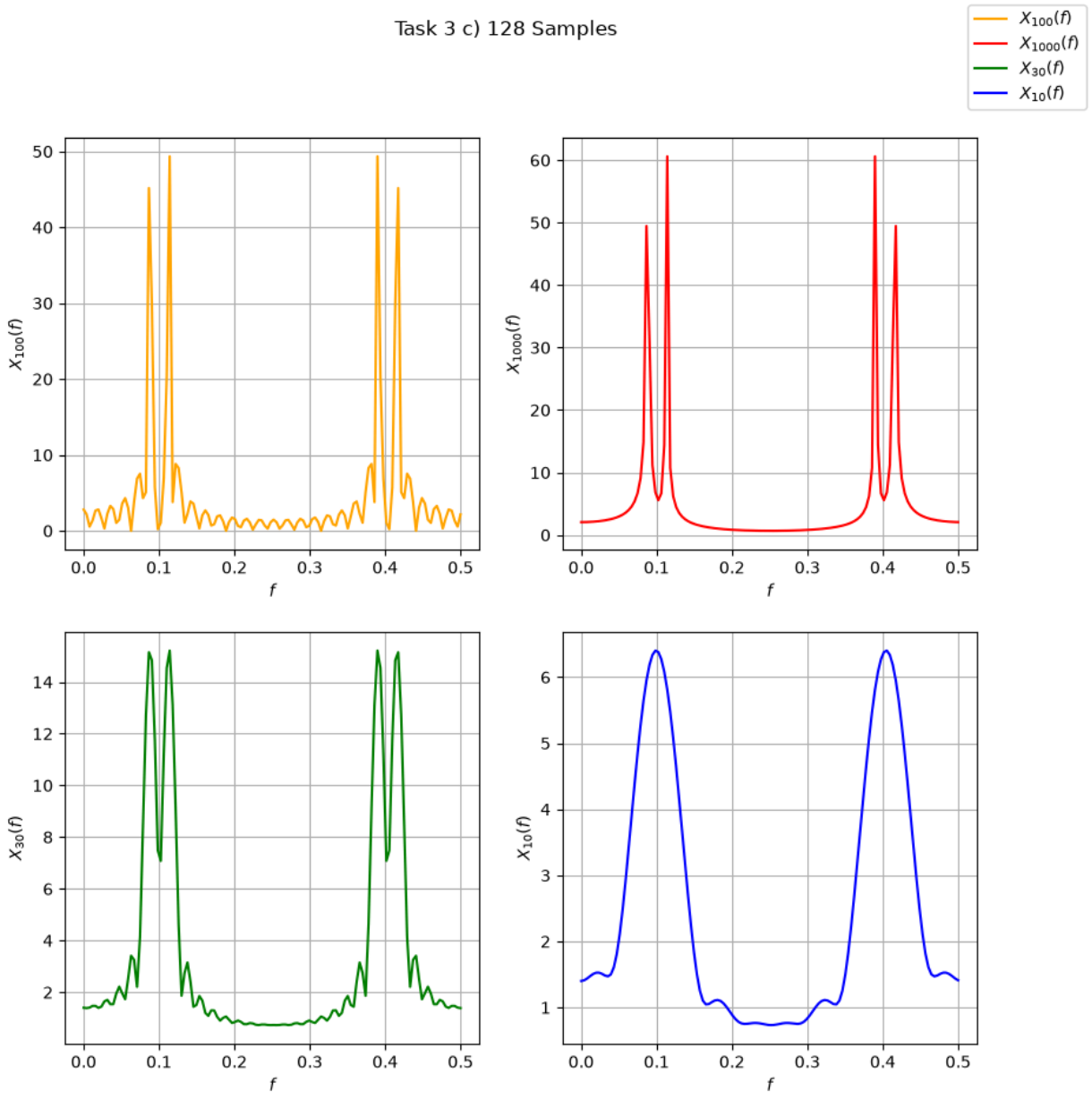


b)



c)





The more samples used for DFT, the flatter the graph/less noice around the peaks.

Problem 4)

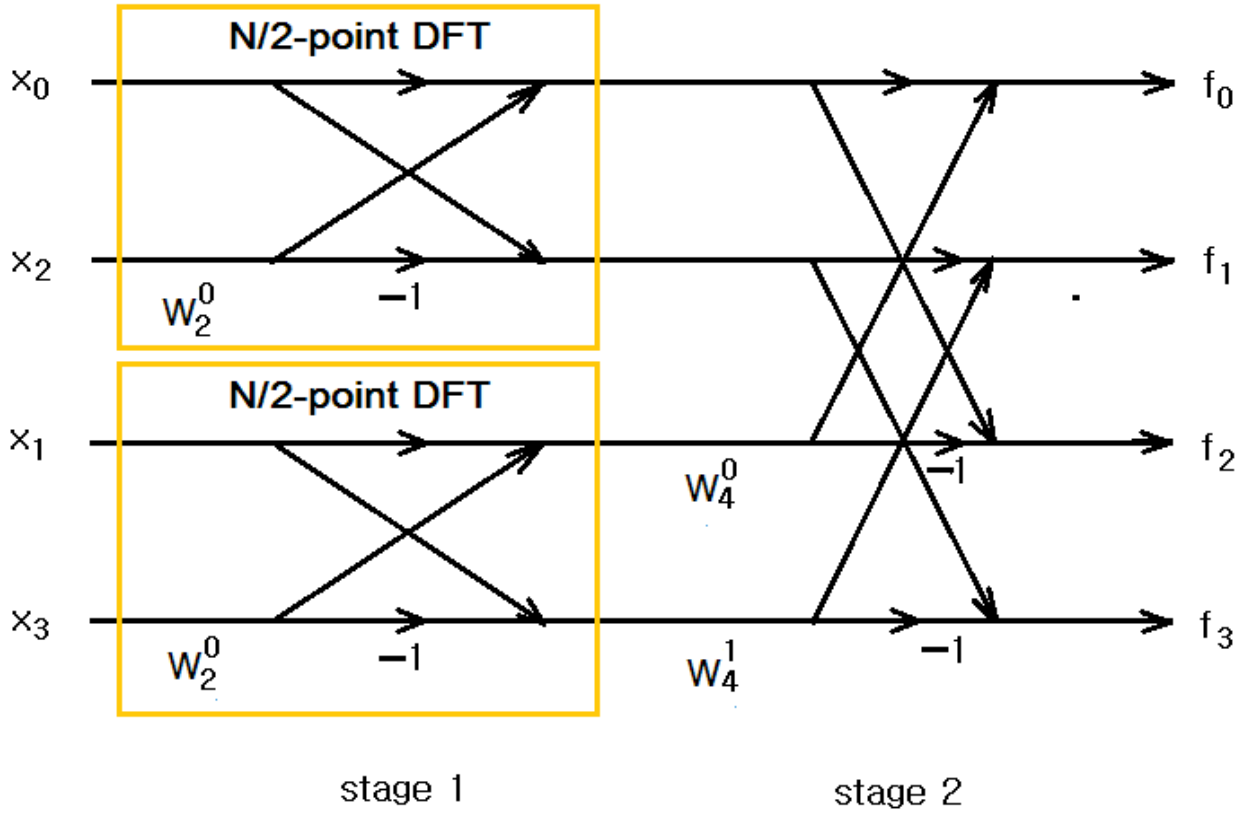
a)

The fast fourier transform is a faster version of DFT. It takes the sampled signal data and represents it in the frequency domain. It takes the compute from a DFT from $O(n^2)$ to $O(n \log n)$ by factorizing the DFT matrix into a product of sparse factors.

b)

Radix-2 FFT is a form of FFT that runs in $O(N \log_2 N)$ if the input size $N = 2^k$. It works by splitting the sequence into smaller DFT's of odds and evens, and summing the different

DFT's with the twiddle factors $W_N^k = e^{-2\pi ik/N}$ as shown in the photo.



c)

The formula for the direct DFT is:

$$X(k) = \sum_{n=0}^{N-1} x[n] W_N^{kn} \quad \text{for } k = 0, 1, \ldots, N-1$$

This means that for each k you need N multiplications, and for all k you then need N^2 multiplications.

d)

Dividing the signal $x[n]$ of length N into the odd and even parts $f_1[n] = x[2n]$ and $f_2[n] = x[2n+1]$ of length $M = N/2$.

$$X(k) = \sum_{n=0}^{N-1} x[n] W_N^{nk} = \sum_{n=0}^{N/2-1} x[2n] W_N^{2k} + \sum_{n=0}^{N/2-1} x[2n+1] W_N^{(2n+1)k}$$

$$\implies \sum_{n=0}^{N/2-1} f_1[n] W_{\frac{N}{2}}^k + W_N^k \sum_{n=0}^{N/2-1} f_2[n] W_{\frac{N}{2}}^k$$

$$\implies X(k) = F_1 + W_N^k F_2(k)$$

Now using the symmetry for the odd:

$$W_N^{(k+M)} = e^{-j2\pi(k+M)/N} = e^{-j2\pi k/N} e^{-j2\pi N/2N} = W_N^k e^{-j\pi} = -W_N^k$$

$$\implies X(k+M) = F_1(k) - W_N^k F_2(k)$$

e)

From previous task, we know that we need N^2 operations for a length of N, meaning F_1 and F_2 need a total of $2M^2 = \frac{N^2}{2}$. Then for each k, the multiplication $W_N^k \cdot F_2(k)$ is done once. This is then used to calculate $X(k)$ and $X(k+M)$ for $k = 0, 1, \ldots, M$ adding another $\frac{N}{2}$ multiplications. This totals out to

$$\frac{N^2}{2} + \frac{N}{2} \approx \frac{N^2}{2} \text{ for large values of N.}$$